Arbeitsjournal

Bezeichnung	Arbeitstyp
R	Recherche
D	Dokumentation
P	Programmieren
O	Organisation

Datum	Arbeit	Typ	Zeit (min)
2025	-	-	-
20.10.	Erstellen eines Dokuments, welches die Grundidee und Inspiration für die Maturaarbeit erklärt. Anfragen der Biologie Lehrpersonen.	D	120
29.11.	Ausfüllen und Abgeben/Hochladen des Vorvertrags.	O	10
16.12.	Erstes Gespräch mit Herrn Leuthardt. Formulieren eines möglichen Ziels der Arbeit.	O	30
30.12.	Erstellen eines Github Repositories für den Code.	P	10
2026	-	-	-
14.01.	Zweites Gespräch mit Herrn Leuthardt. Absprache und Ausfüllen der Arbeitsvereinbarung. Vereinbarung eines Ziels bis zum nächsten Gespräch.	O	30
23.01.	Abgeben/Hochladen der Arbeitsvereinbarung.	O	10
28.01.	Konfigurieren des Repositories für C++ (CMakeLists.txt, src/main.cpp). Importieren einiger	P	120

	header aus einem anderen Projekt. Installieren von Vulkan und GLFW.		
28.01.	Aus Versehen alles unwiderruflich gelöscht. Recherche zu ‘inheritance’ und ‘unions’ in C++.	R	60
29.01.	Wiederholte Konfiguration des Projekts. Erstellen der ersten Elemente: struct Vec<N, T> (speziell: Vec<2, T>, Vec<3, T>, Vec<4, T>), Fundamentale Typen (i8_t, u8_t, i16_t, u16_t, i32_t, u32_t, i64_t, u64_t, f32_t, f64_t, cstr_t) Hilfe: ChatGPT, Reddit, Stack Overflow	P, R	180
30.01.	Weiterarbeiten am Code und Erstellen eines neuen Elements: class Window (erstell und verwalte ein GLFWwindow) Hilfe: ChatGPT, Reddit, Stack Overflow, Vulkan Tutorial (Alexander Overvoorde)	P, R	120
31.01.	Erstellen und Aufarbeiten des Arbeitsjournals	D	60
31.01.	Weiterarbeiten am Code und erstellen neuer Elemente: class VulkanHandle (erstellen und verwalten von Vulkan speziefischen Ressourcen), struct Version (Angeben der einer Version: x.y.z, kompatibel mit Vulkan) Hilfe: ChatGPT, Stack Overflow, Vulkan Tutorial	P, R	270
01.02.	Recherche zur Dokumentation von C++ Code und dokumentation von window.hpp Hilfe: ChatGPT, Stack Overflow	P, D, R	390
02.02.	Dokumentieren des Codes	P, D	240

03.02.	Weiterarbeiten am Code und Hinzufügen neuer Funktionalitäten: Initialisieren und Erstellen von: VkPhysicalDevice, VkQueueFamilies und VkDevice. Hilfe: Vulkan Tutorial	P	180
04.02.	Weiterarbeiten am Code. Versuch, Vulkan graphics- und present queue zu initialisieren. Hinzufügen eines neuen Elements: VkSurface Hilfe: Vulkan Tutorial	P	120
05.02.	Weiterarbeiten am Code. Erfolgreiches Initialisieren des VkSurface und Erstellen der graphics und present queue. Hilfe: Vulkan Tutorial	P	240
03.03.	Weiterarbeiten am Code. Anfang Implementierung der VkSwapchainKHR. Hinzufügen von dafür benötigten Elementen. Hilfe: ChatGPT, Stack Overflow, Vulkan Tutorial	P, R	120
04.03.	Weiterarbeiten am Code. Erfolgreiches Initialisieren der VkSwapchainKHR und Anfang Implementierung der VkPipeline. Hinzufügen dafür benötigter Elemente. Hilfe: Vulkan Tutorial	P, R	180
05.03.	Weiterarbeiten am Code. Erfolgreiches Initialisieren der graphics pipeline und Implementieren der VkCommandBuffer Funktionalität und entsprechender Elemente. Erstmaliges erfolgreiches Ausführen des Programms und Rendern eines einzelnen Dreiecks.	P, R	500

	Optimierung durch «frames-in-flight» Funktionalität. Hilfe: ChatGPT, Vulkan Tutorial		
07.03.	Weiterarbeiten am Code. Anfang Implementierung eines Vertex Buffers. Hilfe: Vulkan Tutorial	P, R	180
08.03.	Weiterarbeiten am Code. Wechseln von GLFW zu SDL3. Hinzufügen eines logging und debugging Systems für Erstellung von Ressourcen und Fehlern. Hilfe: ChatGPT, Stack Overflow, Vulkan Tutorial	P, R	240
09.03.	Weiterarbeiten am Code. Erfolgreiches Implementieren des Vertex Buffers und Arbeit an verschiedenen Shadern. Hilfe: Vulkan Tutorial	P, R	300
10.03.	Weiterarbeiten am Code. Hinzufügen einer Vulkan push constant für die Bildschirm Auflösung (benötigt von den Shadern). Erfolgreiches Implementieren eines funktionierenden Kreis Shaders. Hilfe: Vulkan Tutorial	P, R	180
11.03.	Weiterarbeiten am Code. Experimentieren mit verschiedenen Fragment-Shadern und Implementieren von color blending Hilfe: ChatGPT, Vulkan Tutorial	P, R	120
12.03.	Weiterarbeiten am Code. Hinzufügen einer Funktion (DrawCircle()) zu class Window, um einen Kreis zu zeichnen. Hat noch ein paar Probleme. Hilfe: ChatGPT, Vulkan Tutorial	P	120

03.04.	Weiterarbeiten am Code. DrawCircle() funktioniert und erste Tests. Kreis-Renderer ist soweit fertig. Hilfe: ChatGPT, Vulkan Tutorial	P, R, D	180
04.04.	Weiterarbeiten am Code. Hinzufügen einer Funktion (DrawRectangle()) zu class Window, um ein Rechteck zu zeichnen. Hinzufügen der Funktionalität, Kreise und Rechtecke mit verschiedenen Shader-Effekten zu zeichnen. Hilfe: ChatGPT, Stack Overflow, Vulkan Tutorial	P	180
02.05.	Weiterarbeiten am Code. Bisherige chemische Simulationselemente gelöscht und an biologischen Zellbestandteilen angefangen. Hinzufügen einer «Particle» und einer «Compound» class und Anfang der Funktionalität von Phospholipiden.	P, R	180
03.05.	Erste Erfolge mit den Phospholipiden. Die hydrophoben Enden gruppieren sich und es entstehen kleine Gruppen. Aufgrund von Problemen mit der Rotationsberechnung bleiben die Teilchen nicht parallel zueinander → noch keine längere Kette möglich. Hinzufügen einiger anpassbaren Konstanten, um das Verhalten der Teilchen zu verändern und Herumprobieren. Hilfe: ChatGPT	P, R	420
05.05.	Verändern der Interaktion von hydrophilen und hydrophoben Teilchen. Alt: Lineare Skalierung der Anziehung/Abstossung anhand der Distanz. Kollision bzw. Überlappung muss zusätzlich erkannt und verhindert werden. Neu: Anziehung/Abstossung nach dem «Lennard-Jones-Potential». Verhindert Kollision durch exponentielle Abstossung auf kleine Distanz.	P, R	240

	Neuer Ansatz löst das Problem mit der Rotationsberechnung und zum ersten Mal entstehen längere Ketten (mehr als 10 Phospholipide) Hilfe: ChatGPT, Wikipedia zum Lennard-Jones Potential		
06.05. – 16.05.	Verändern der Konstanten, um das Entstehen einer einzelnen Kette ohne Hilfe zu ermöglichen. Es gibt Verbesserungen. Veränderte Werte: maximale Interaktionsdistanz von Partikeln, hydrophobe Anziehung, hydrophile Anziehung, Abstossung zwischen hydrophob-hydrophil, Konstanten für Lineare- und Winkelgeschwindigkeit, Geschwindigkeitsdämpfung über Zeit. Weiterhin Limitieren der Geschwindigkeitsdämpfung auf minimale Geschwindigkeiten, um das Stoppen aller Teilchen zu verhindern. Gesamtgeschwindigkeit aller Teilchen verringert sich trotzdem langsam → Beschleunigen aller Teilchen, sobald die Gesamtgeschwindigkeit einen bestimmten Wert unterschreitet Hilfe: ChatGPT	P, R	480
17.05.	Aufarbeiten des Journals der letzten Tage. Mit Hilfe eines Skriptes die Anzahl an Linien in allen Quelldateien zusammengezählt: ca. 7200. Hilfe: ChatGPT	D	30

14.06.	Überlegungen zur Implementierung von Proteinen, DNA/RNA und ihrer Interaktion. Weitere Versuche an der Phospholipid Doppelmembran mit verschiedenen Werten.	P	45
15.06.	Implementieren eines Systems um einfacher auf der Simulationsfläche zu Navigieren. Zoom durch Scrollen mit der Maus und Bewegen mit den Pfeiltasten.	P	120
17.06.	Letztes Gespräch mit Herrn Leuthardt vor den Sommerferien.	O	15
21.06.	Weiterarbeiten an der Implementierung von Proteinen, RNA und DNA. Hilfe: ChatGPT, Stack Overflow	P	60
22.06.	Aufräumen und Strukturieren des Programms. Verlegen aller Elemente in main.cpp (ausser die main() Funktion) in header-Dateien. Globale Variablen und Konstanten → parameters.hpp, Particle-Klasse → particle.hpp, Compound-Klasse und dazu gehörende Elemente → compound.hpp, restliche Funktionen → functions.hpp. Hinzufügen einer header-Datei für alle möglichen Verbindungen in der Simulation (compound_structures.hpp). Mit Hilfe des Skriptes die Anzahl an Linien in allen Quelldateien nochmals zusammengezählt: ca. 7500.	P	60

23.06.	Code und Journal mit GitHub synchronisiert. Hinzufügen einer Timer-Klasse (Singleton), um zeitspezifische Daten zu verwalten und die, in der Simulation verstrichene Zeit seit einem bestimmten Frame zu berechnen (wichtig für spätere Abläufe von Proteinen). Hinzufügen einer third-party-library, um die Simulationsdaten im JSON-Format zu speichern. Hilfe: ChatGPT	P	300
24.06.	Hinzufügen eines Systems, das alle wichtigen Daten einer laufenden Simulation speichert und gespeicherte Daten wieder laden kann. Erstellen einer Hashmap, die alle möglichen Verbindungen speichert und genutzt werden kann, um ein neues Element mit nur dessen Namen zu erstellen (Bsp.: «phospholipid» und «protein_1»). Erste Prototypen von Funktionen, die spezielle Interaktionen von Verbindungen ermöglichen (Transkription, Translation). Erste Erfolge beim Simulieren der Proteinbiosynthese. Hilfe: ChatGPT, Stack Overflow	P	480
25.06.	Verbessern des Interaktionssystems und Hinzufügen einer Funktion für DNA-Replikation. Hinzufügen einer weiteren third-party-library, um das Programm mit einem Thread Pool zu parallelisieren. Weitere Verbesserungen und Problembehebungen im Code.	P	600

	Erstellen eines speziellen Elements, das alles löscht, was es berührt, um die Simulation besser kontrollieren zu können. Implementierung der Parallelisierung und Verbessern der Geschwindigkeit des Programms. Bei ca. 500 Verbindungen: ca. 38FPS (ohne Parallelisierung) → ca. 75FPS (mit Parallelisierung)		
26.06.	Verbessern der Interaktionen für Transkription, Translation und DNA-Replikation. Proteine bewegen sich jetzt am DNA/RNA-Strang entlang und erstellen neue Verbindungen, sobald sie am Ende ankommen. Hinzufügen einer optionalen Lebenszeit für Verbindungen. Umstrukturierung des Projekts in Module, die separat kompiliert werden → nach einer Änderung muss nicht der gesamte Code nochmals kompiliert werden. Rückgängig machen der Parallelisierung, da globale Variablen nicht einfach so von mehreren Threads verändert werden dürfen. Code für Parallelisierung ist auskommentiert. Beheben von diversen Fehlern und Testen verschiedener Konstellationen und der Speicherfunktion. Hilfe: ChatGPT	P	360
22.07.	Umstrukturieren des GitHub Repositories für Übertragung an einen neuen Account (j-goth). Erstellen des Dokuments für die Dokumentation. Beginn der Dokumentation zum Produkt.	O, D	30

30.07.	Kleine Änderung der Dateien für das Erstellen von Game Elementen und Gruppieren zusammenhängender Dateien in eigene Ordner. Umstrukturieren des CMake Setups, da beim Kompilieren bzw. Linken Probleme auftreten. Hilfe: ChatGPT	P, O	60
06.08.	Aufarbeiten des Journals der letzten Tage/Wochen. Vorbereiten des Projekts, um es zum neuen GitHub Account zu transferieren. Lösen des Linker Problems durch das Entfernen eines inkorrekten «inline» bei der Definition einer Funktion.	P, O	30
07.08.	Weiterschreiben der Dokumentation am Kapitel Produkt.	D	180
08.08.	Weiterschreiben der Dokumentation am Kapitel Produkt.	D	180
29.08.	Weiterschreiben der Dokumentation und Anfang der Einleitung.	D	60
30.08.	Beenden des Textteils der Rohfassung der Dokumentation. Provisorische Abgabe der unfertigen Version mit fehlenden Bildern und Quellen, zur Sicherheit.	D	500
31.08.	Fertigstellen der Rohfassung und richtige Abgabe an Herr Leuthardt.	D	180
29.09.	Anfang der Überarbeitung der Dokumentation anhand der Rückmeldung.	D	30
30.09.	Weiterarbeiten an der Überarbeitung.	D	30

03.10.	Fertigstellen der Überarbeitung des Textteils der Dokumentation. Anpassen des Programms für Kompilierung auf Windows. Reduzieren der benötigten Systeme durch statisches linken von SDL3 und nlohmann-json → um die kompilierte Datei auszuführen muss nur noch Vulkan im installiert sein. Hinzufügen von SDL3 als Submodul des Git Projekts und anpassen der CMake Konfiguration. Hilfe: ChatGPT	D, P	300
04.10.	Überarbeitung der restlichen Elemente der Dokumentation (Bilder, Layout, etc.). Versuch, Virtual Machines mit Windows 11 und MacOS zu erstellen, um das Programm auf allen Systemen testen zu können. Erste Wahl war Oracle VirtualBox, ist wegen fehlendem Vulkan Support jedoch unpassend. Wechseln zu qemu/kvm, jedoch auch kein Erfolg.	D, P	270
05.10.	Versuch, die CMake Konfiguration anzupassen, sodass die Vulkan Shader nur dann automatisch kompiliert werden, wenn sie bearbeitet wurden. Die versuchten Ansätze haben alle noch Fehler. Aufarbeiten des Journals der letzten Woche. Hilfe: ChatGPT	D, P	210
06.10.	Hinzufügen der Möglichkeit, Verbindungen in einer JSON-Datei zu bearbeiten und hinzufügen, damit das Programm nicht ständig neu kompiliert werden muss. Hilfe: ChatGPT	P	360

07.10.	Hinzufügen eines Terminal-Interfaces, über das man Parameter der Simulation ausgeben und verändern kann.	P	600
08.10.	Überarbeiten der Dokumentation und Einfügen aller Quellen.	D	420
09.10.	Überarbeiten der Dokumentation. Deutsche Version ist fertig. Schreiben einer kurzen Zusammenfassung für die Bewerbung beim BioValley College Award. Übersetzen der gesamten Dokumentation in Englisch mit DeepL. Hilfe: DeepL	D	480
10.10.	Überarbeiten der englischen Version der Arbeit. Fertigstellen der Zusammenfassung und ebenfalls mit DeepL in Englisch übersetzten. Hilfe: DeepL	D	240
11.10.	Fertigstellen und Überarbeiten des Journals. Übersetzen in Englisch. Veröffentlichen der gesamten Arbeit auf GitHub. Kompilieren und Hochladen des Programms für Windows und Linux (GitHub Release). Finale Abgabe. Hilfe: DeepL	D	300